Claude Agentic Workflows

Four production-style AI workflows built on the Claude API

What this is:

This is a portfolio of four small but complete systems that show how to use a large language model reliably in real work: routing requests, reviewing code, processing large datasets, and researching a question with sources.

Each one includes the surrounding engineering that makes AI trustworthy: validation, safety checks, automated tests, and a way to measure quality.

The four patterns at a glance

Project	Pattern	In one sentence
Customer Support Router	Routing	Sorts incoming tickets and answers each with the right specialist, escalating risky ones to people.
Code Review Workflow	Prompt chaining	Reviews code in steps, checking each finding before it reaches the report.
Data Processing Pipeline	Parallelization	Turns thousands of text records into structured data using many calls at once.
Research Assistant Agent	Autonomous agent	Searches, reads and writes a report where every claim is backed by a checked quote.

Principles shared by all four

- **Code sets the rules, the model does the thinking.** Limits, validation and final decisions are ordinary code, so they behave the same way every time.
- **Fail safe.** If the AI is unsure or returns something malformed, the system asks a person or says so clearly instead of guessing.
- **Untrusted text stays data.** Tickets, code and documents may contain hidden instructions; they are flagged and never obeyed. Personal data and secrets are masked before any model sees them.
- **Measured, not assumed.** Each project has a labelled test set and metrics, including safety metrics.
- **Testable without an API key.** A built-in mock lets the whole suite run anywhere, including automated checks on GitHub.

Shared library: agentic-core

A small common layer used by all projects: a typed async Claude client with retries and token counting, a mock client for tests, redaction of personal data and secrets, injection detection, an audit log, and evaluation metrics.

1. Customer Support Router

Pattern: Routing

What it does

Takes a customer ticket, decides what kind it is (billing, technical, account, sales, general or escalation), and drafts a reply using a prompt written for that specialty. Sensitive or uncertain tickets go to a human instead.

> **Ticket → Mask personal data → Classify (fast model) → Safety gate → Specialist reply → Audit log**

How it works

- The classifier is forced to answer in a fixed structure, which is then validated in code.
- A cheap, fast model classifies; a stronger model only writes replies for tickets that pass the gate.
- Each specialist prompt lives in its own file so it can be reviewed and changed without touching code.

Safeguards

- Legal, security, press and regulatory tickets are never answered automatically.
- Low confidence, invalid output or a suspected injection attempt goes to a human.
- Emails, card numbers and phone numbers are masked before the model or the logs see them.

How it is tested

Eval on 24 labelled tickets measures accuracy, escalation recall (target 100%), accuracy of auto-answered tickets and hand-off rate. 11 unit tests cover each safeguard.

2. Code Review Workflow

Pattern: Prompt chaining

What it does

Reads a source file and produces a review: a verdict (approve, request changes, block), a list of issues with severity, and suggested fixes.

Mask secrets → Analyze → Check evidence → Verify (second pass) → Suggest fixes → Report

How it works

- Each step feeds the next, with a code check in between so mistakes do not compound.
- The evidence check rejects any finding whose line number or quoted code is not really in the file.
- A second, skeptical pass removes false alarms. Clean code stops early to save cost.
- The final verdict comes from a fixed severity policy, not from the model. A blocking verdict returns a non-zero exit code so it can stop a build.

Safeguards

- Hard-coded passwords and keys are masked and the author is warned to rotate them.
- Comments that try to instruct an AI reviewer are flagged and ignored.
- If a step fails, the report says so rather than silently passing.

How it is tested

Eval on sample files with known seeded bugs measures line-level precision and recall, false alarms on clean code, and secret leakage. 12 unit tests.

3. Data Processing Pipeline

Pattern: Parallelization

What it does

Takes a large file of customer feedback and extracts sentiment, topics and a short summary for every record, running many model calls at the same time.

> **Read and clean → Batches of 20 → Worker pool → Extract + retry → Vote if unsure → Save + report**

How it works

- Records are batched about 20 per call, cutting calls roughly tenfold, and processed by a bounded pool of workers with a request-rate limit.
- Temporary errors are retried with growing, randomized delays.
- If a batch fails, it is split in half until the single bad record is found and set aside in a failures file.
- Uncertain records are re-run twice and decided by majority; ties go to a person.
- Progress is saved after every batch, so a crashed run resumes where it stopped.

Safeguards

- Malformed lines and duplicates are counted, never silently dropped.
- Every model answer is checked item by item; invented or invalid items are discarded and retried.
- Personal data is masked before the model sees it.

How it is tested

Eval on 300 labelled reviews (including sarcasm) measures accuracy, trust in auto-accepted records and review rate. 13 tests inject rate limits, poison records, partial answers and a crash mid-write.

4. Research Assistant Agent

Pattern: Autonomous agent

What it does

Given a question, plans sub-questions, searches a set of documents, reads the best ones, records evidence, and writes a report in which every statement carries a citation.

Plan → Search → Read → Record evidence → Write report → Check citations

How it works

- The model chooses which tools to use, in what order, and when it has enough. Several searches or reads can run at once.
- A fact can only be recorded with a quote copied exactly from a document the agent actually read. Wrong quotes are bounced back so the agent corrects itself.
- The report is written only from the recorded evidence. Citations that do not match a source are marked, and uncited sentences are counted.
- Conflicting sources must be shown side by side.

Safeguards

- Hard limits on steps, tool calls and tokens; the report says if a limit stopped the run.
- No evidence means no report, only a clear note.
- Documents are untrusted data. The tools are read-only, and web access blocks private network addresses.

How it is tested

Eval on four questions over a synthetic document set (including a document that tries to hijack the agent) measures fact recall, citation integrity and injection resistance. 18 unit tests.

How to run it

You need Python 3.10 or newer.

No API key is required to run the tests.

```
python run_tests.py
cd customer-support-router
pip install -e ../agentic-core -e .
support-router --offline eval data/tickets_golden.jsonl
```

To use real Claude models, copy `.env.example` to `.env` in a project folder, add your API key, and remove `--offline`.

Take Home:

- Choosing the right agentic pattern for a problem, and knowing when each is appropriate.
- Production concerns: reliability, cost control, security, privacy, auditability and human oversight.
- Evaluation-driven development with automated tests and CI.